

Harness Engineering — резюме в одной странице

по материалу «Production Agent Engineering Practice 2026: Agent = Model + Harness» (август 2026)

Формула

Agent = Model + Harness. Модель даёт интеллект. Harness даёт всё остальное. Доказательная база: +43.64 пп на GAIA (Claude Sonnet 4.5 без смены модели), +25 рангов на Terminal Bench 2.0 (LangChain), миллион строк production-кода без ручного кода (OpenAI Codex). Это **больше**, чем даёт разница между большинством смежных поколений моделей.

Шестислойная архитектура

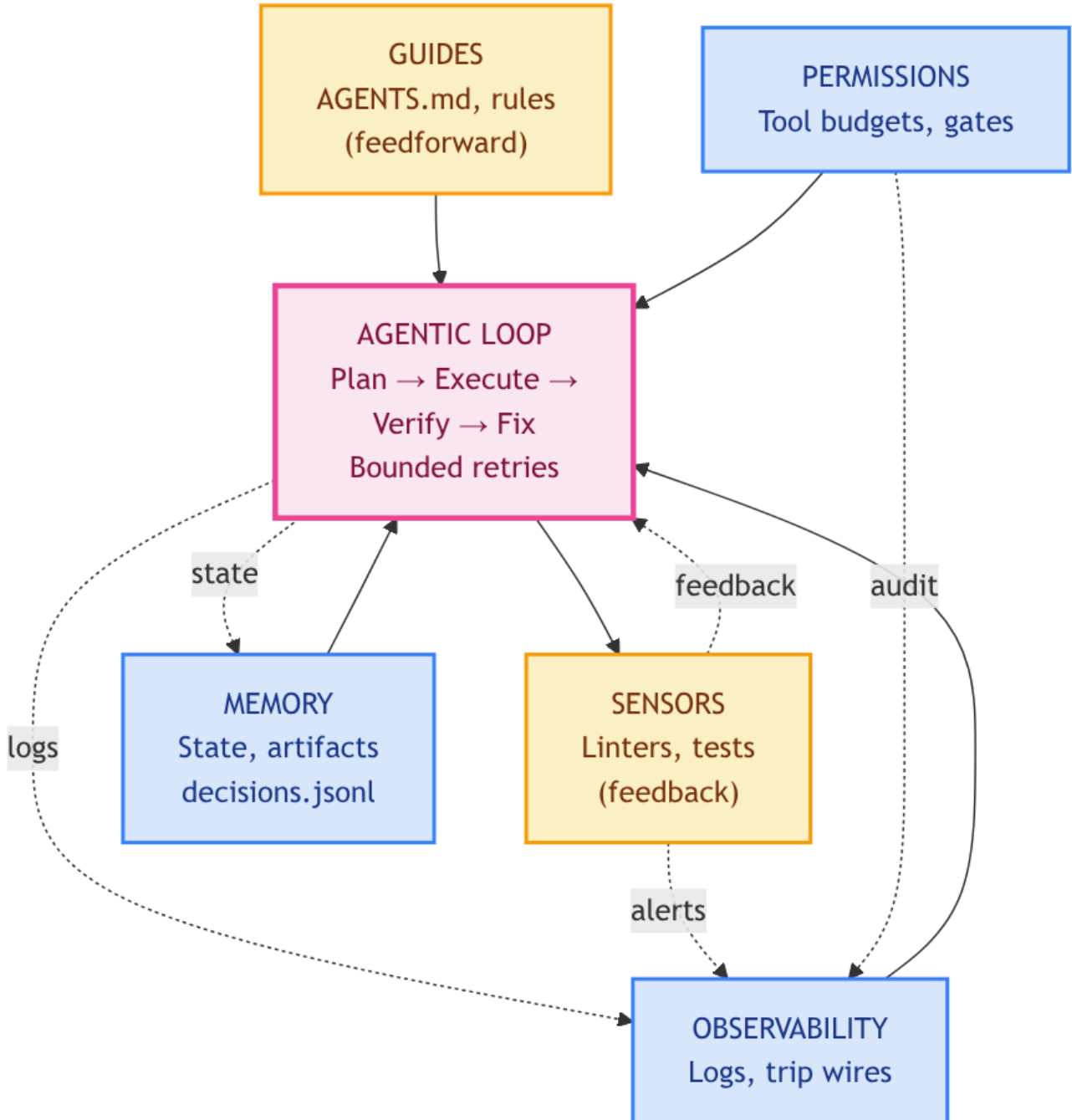


Рис. 1. Steering loop (Guides + Loop + Sensors) и runtime foundation (Memory + Permissions + Observability).

Верхний ряд (steering loop): **Guides** (feedforward, читаются до запуска — AGENTS.md, rule files) → **Agentic Loop** (Plan → Execute → Verify → Fix с bounded retries) → **Sensors** (feedback после запуска — linters, тесты, валидаторы).

Нижний ряд (runtime foundation): **Memory** (state files, artifacts, decisions.jsonl), **Permissions** (tool budgets, write limits, approval gates), **Observability** (trace logs, trip wires).

Ratchet Principle — главное новшество

Каждая ошибка → **постоянное** улучшение системы, а не разовый фикс. Цикл из 6 шагов:

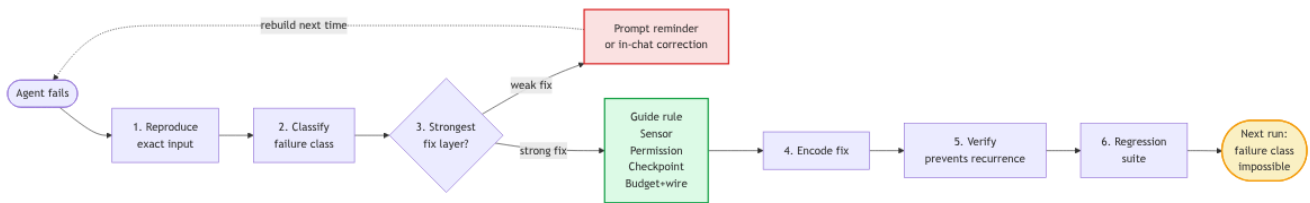


Рис. 2. Ratchet loop — каждый фикс на сильном слое (guide / sensor / permission) делает класс ошибок структурно невозможным.

Правило Lauren Tan (Cursor): один и тот же review-комментарий 3 раза подряд → структурное ограничение, а не вкусовщина. Первый раз → guide rule. Второй → проверка, что читается. Третий → sensor, блокирующий нарушение. Четвёртого быть не должно.

Сигнал зрелости: зрелый harness добавляет 1 правило в неделю, не 5 в день. Замедление роста guide-файла = основные failure-классы закрыты.

Ключевые принципы (компактно)

Computational > Inferential. Сначала детерминированные проверки (линтеры, тесты). LLM-as-judge — только когда семантику нельзя выразить правилом. Линтер = 0 рублей за прогон. LLM-judge 50 раз/день = \$500/день.

Guides as organizational memory. Без guide-файла каждая сессия стартует с нуля. Каждое исправление, сделанное в чате, теряется с концом сессии. Guide-файл = постоянство.

Filesystem как memory. plan.md + decisions.md + progress.json — дешевле и надёжнее векторных БД для большинства workflow.

Когда harness НЕ нужен

Одноразовые вопросы, creative brainstorming, exploratory разговоры. Harness окупается, когда: workflow повторяется, ошибки имеют последствия, агент работает без supervision, state должен переживать сессию. **Тест:** узнаешь ли за час, что агент начал ломать выходные? Если да — нужен sensor. Нужно ли объяснять контекст заново? Если да — нужна memory. Создаёт ли ошибка внешние последствия? Если да — нужны permissions.

Три эпохи AI-инженерии

Эпоха	Фокус	Оптимизирует	Граница
Prompt Eng	Один ход	Фразировка, CoT	Один turn
Context Eng (2025)	Что видит модель	RAG, MCP, memory	System context
Harness Eng (2026)	Что может модель	Execution, safety	Весь рантайм

Build Path: 7 дней до MVP

День	Что строить	Exit-test
1	AGENTS.md с build/test/lint	Агент запускает все 3 правильно
2	+3 guide rules из прошлых ошибок	Избегает 3 anti-pattern'a
3	+1 computational sensor (test suite)	Прогоняет тесты после каждого изменения
4	+agentic loop + retry budget	Ретраит, потом эскалирует
5	+JSON checkpoint на filesystem	Восстанавливается после рестарта
6	+permissions + cost budget	Не может превысить scope
7	+structured logging + trip wire	Trip wire срабатывает на симуляции

Не аффилировано с Google, OpenAI, Anthropic или HashiCorp. Первоисточники: М. Hashimoto (5 фев 2026), OpenAI/R. Lopopolo (11 фев 2026), Martin Fowler/B. Böckeler (2 апр 2026), LangChain Eng (17 фев 2026), L. Tan/Cursor.